aa279b/orbital.py

```python
import numpy as np
from numpy.linalg import norm
from scipy.integrate import solve_ivp
from scipy.optimize import fsolve
from scipy.interpolate import CubicSpline
import matplotlib.pyplot as plt
from lamberthub import vallado2013
import plotly.graph_objects as go
from tqdm import tqdm

from constants import DEG_TO_RAD, MU_EARTH, W_EARTH, R_EARTH, AU_TO_KM, MU_SUN
from angle import dcm_x, dcm_z
from spacetime import greg_to_jd
from matlab.engine import MatlabEngine



class OrbitalFlyby:
    def __init__(self, name: str, datetime_ymdhms_utc: tuple[int,int,int,int,int,float], planet_num: int = None):
        self.name = name
        self.date = datetime_ymdhms_utc
        self.planet_num = planet_num
        self.jd = greg_to_jd(*datetime_ymdhms_utc)

        self.hyperbolic_turning_angle = None
        self.hyperbolic_eccentricity = None

        self.v_departure_HCI, self.v_arrival_HCI = None, None
        self.v_departure_PCI, self.v_arrival_PCI = None, None

    def add_rv_HCI(self, r_HCI: np.ndarray, v_HCI: np.ndarray):
        self.r_HCI, self.v_HCI = r_HCI, v_HCI

def rv_HCI(planet_num: int, jd: float, eng: MatlabEngine):
    """Heliocentric position and velocity using https://ssd.jpl.nasa.gov/planets/approx_pos.html

    From `AA279j2000_planetary_elements` matlab:
    elements:  row vector of J2000 orbital elements for "planet_id"
    - a             semimajor axis [AU]
    - e             eccentricity
    - i             inclination [deg]
    - OMEGA         longitude of the ascending node [deg]
    - omega_bar     longitude of perihelion [deg]
    - L             mean longitude [deg]

    Parameters
```

```
    ----------
    planet_num : int
        1 (Mercury) to 9 (Pluto) BUT further ones beyond Mars technically require more coefficients
    jd : float
        Julian Date

    Returns
    ----------
    r_HCI : np.ndarray (3,)
        Heliocentric position
    v_HCI : np.ndarray (3,)
        Heliocentric velocity
    """

    assert planet_num >= 1
    assert planet_num <= 9

    tt_jd = (jd - 2451545.0) / 36525.0

    elements, rates = eng.AA279j2000_planetary_elements(planet_num, nargout=2)


    #  1. Value of six elements (angles in degrees)
    a_au, e, i_deg, raan_deg, w_bar_deg, L_deg = [
        x + dx*tt_jd for x,dx in zip(elements[0], rates[0])
    ]

    #  2. aop and M
    aop = (w_bar_deg - raan_deg) * DEG_TO_RAD
    M = (L_deg - w_bar_deg) * DEG_TO_RAD

    #  Just solve (from prev homework)
    r_HCI, v_HCI = rv_from_coe(
        a = a_au * AU_TO_KM,
        e = e,
        i = i_deg * DEG_TO_RAD,
        raan = raan_deg * DEG_TO_RAD,
        aop = aop,
        M = M,
        mu = MU_SUN
    )

    return r_HCI, v_HCI

def kepler(M: float, e: float):
    """Vallado 4e eq. 2-4 p. 45 (pdf p. 72)

    Parameters
    ----------
```

```
    M : float
        Mean anomaly [rad]
    e : float
        Eccentricity [0, 1)

    Returns
    -------
    E : float
        Eccentric anomaly [rad]
    """
    E = fsolve(lambda E: E - e * np.sin(E) - M, M)[0]
    return E

############################################################################
#                    RV <-- COE
############################################################################

def rv_from_coe(a: float, e: float, i: float, raan: float, aop: float, M: float, mu: float) -> tuple[np.ndarray, np.ndarray]:
    """Convert classical orbital elements to inertial position and velocity (not just ECI)

    Vallado 4e Algorithm 10 p. 118 (pdf p. 145)

    Parameters
    ----------
    a : float
        Semimajor axis [km]
    e : float
        Eccentricity
    i, raan, aop, M : float
        Inclination, RAAN, argument of periapsis, mean anomaly [rad]
    mu : float
        Gravitational parameter of central body [km3/s2]

    Returns
    -------
    r_I, v_I : np.ndarray (3,)
        Inertial position [km] and velocity [km/s]
    """

    # semiparameter - Vallado 4e p. 96 (pdf p. 123)
    p = a * (1 - e * e)

    E = kepler(M, e)

    # true anomaly - Vallado 4e. eq. 2-10 p. 47 (pdf p. 74)
    nu = 2 * np.arctan2(np.sqrt(1 + e) * np.sin(E / 2), np.sqrt(1 - e) * np.cos(E / 2))
    r_PQW = np.array([p * np.cos(nu), p * np.sin(nu), 0]) / (1 + e * np.cos(nu))

```

```
    v_PQW = np.array([-np.sin(nu), e + np.cos(nu), 0]) * np.sqrt(mu / p)

    dcm_I_wrt_PQW = dcm_z(-raan) @ dcm_x(-i) @ dcm_z(-aop)
    r_I = dcm_I_wrt_PQW @ r_PQW
    v_I = dcm_I_wrt_PQW @ v_PQW

    return r_I, v_I

################################################################################
#                   PCPF <--> LLA
################################################################################

def PCPF_from_lla(lat: float, lon: float, alt_km = 0.0, r_body_km = R_EARTH, deg = True):
    """Convert latitude, longitude, and altitude to planet-centered, planet-fixed.
    Spherical body.

    Parameters
    ----------
    lat, lon : float
        Lat,lon [deg if deg=True, else rad]
    alt_km : float
        Altitude above SPHERICAL surface [km]
    r_body_km : float
        Body radius [km]
    deg : bool
        Whether input angles are in degrees

    Returns
    -------
    r_PCPF : np.ndarray (3,)
        Body-fixed position [km]
    """
    if deg:
        lat *= DEG_TO_RAD
        lon *= DEG_TO_RAD

    rho = r_body_km + alt_km

    r_PCPF = np.array([
        rho * np.cos(lat) * np.cos(lon),
        rho * np.cos(lat) * np.sin(lon),
        rho * np.sin(lat)
    ])

    return r_PCPF

################################################################################
#                   PCI <--> PCPF
################################################################################
```

```python

def calc_theta_g(t_utc: float, theta_ref: float, t_ref: float, w_body = W_EARTH):
    """Simple rotation from Greenwich rotation angle assuming constant rotation rate.

    Parameters
    ----------
    t_utc, t_ref : float
        Current and reference UTC timestamps [s]
    theta_ref : float
        Earth rotation angle at reference time [rad]
    w_body : float, optional
        default `W_EARTH` [rad/s]

    Returns
    ------
    theta_g : float
        Greenwich rotation angle [rad], wrapped to [0, 2*pi)
    """
    return (theta_ref + w_body * (t_utc - t_ref)) % (2 * np.pi)

def calc_PCI_from_PCPF(r_PCPF: np.ndarray, v_PCPF: np.ndarray, t_utc: float,
theta_ref: float, t_ref: float, w_body = W_EARTH):
    """Convert rotating-frame position and velocity to inertial coordinates.

    Somewhat simplified version of itrf-gcrs in Vallado 4e Section 3.7 pp. 203

    Uses a constant body rotation rate, neglecting precession and nutation.

    Parameters
    ----------
    r_PCPF, v_PCPF : np.ndarray (3,)
        Position [km] and rotating-frame velocity [km/s]
    t_utc, t_ref : float
        Current and reference UTC timestamps [s]
    theta_ref : float
        Rotation angle at reference epoch [rad]
    w_body : float
        Body rotation rate [rad/s]

    Returns
    -------
    r_PCI, v_PCI : np.ndarray (3,)
        Inertial position [km] and velocity [km/s]
    """
    # Earth rotation angle
    theta_PCPF_wrt_PCI = calc_theta_g(t_utc, theta_ref, t_ref, w_body)

    dcm_PCI_wrt_PCPF = dcm_z(theta_PCPF_wrt_PCI).T

    r_PCI = dcm_PCI_wrt_PCPF @ r_PCPF
```

```
    v_PCI = dcm_PCI_wrt_PCPF @ (v_PCPF + np.cross([0,0,w_body], r_PCPF))

    return r_PCI, v_PCI

################################################################################
#                   PCI <--> LLA
################################################################################

def launch_PCI(t_arr_utc: np.ndarray, lat: float, lon: float, alt: float, theta_ref: float, t_ref: float, r_body_km = R_EARTH, w_body = W_EARTH, deg = True):
    """Calculate inertial position and velocity of a stationary launch site over time on sphere

    Parameters
    ----------
    t_arr_utc : np.ndarray (N,)
        UTC timestamps [s]
    lat, lon : float
        Lat,lon [deg if deg=True, else rad]
    alt : float
        Altitude above SPHERICAL surface [km]
    theta_ref : float
        Greenwich angle at reference epoch [rad]
    t_ref : float
        Reference UTC timestamp [s]
    r_body_km : float
        Body radius [km]
    w_body : float
        Body rotation rate [rad/s]
    deg : bool
        Whether input angles are in degrees

    Returns
    -------
    launch_r_PCI, launch_v_PCI : np.ndarray (N, 3)
        Inertial positions [km] and velocities [km/s]
    """
    launch_r_PCPF = PCPF_from_lla(lat, lon, alt, r_body_km, deg)
    launch_v_PCPF = np.zeros(3)

    launch_arr_PCI = np.array([calc_PCI_from_PCPF(launch_r_PCPF, launch_v_PCPF, t, theta_ref, t_ref, w_body) for t in t_arr_utc])
    launch_r_PCI = launch_arr_PCI[:,0,:]
    launch_v_PCI = launch_arr_PCI[:,1,:]
    return launch_r_PCI, launch_v_PCI

################################################################################
#                   Sanity
################################################################################

# Vallado 4e example 2-6 p. 119 (pdf p. 146)
```

```python
assert np.allclose(
    np.array(
        rv_from_coe(
            a=36126.6428348052,
            e=0.83285,
            i=87.87 * DEG_TO_RAD,
            raan=227.89 * DEG_TO_RAD,
            aop=53.38 * DEG_TO_RAD,
            M=7.60494 * DEG_TO_RAD,
            mu=MU_EARTH,
        )
    ),
    np.array([[6525.344, 6861.535, 6449.125], [4.902276, 5.533124, -1.975709]]),
)


```